CACHING · 1

What is a cache **hit**?

CACHING · 2

What is a cache **miss**?

CACHING · 3

What does a **TTL** set?

CACHING · 4

Why give every key a TTL, even with invalidation on write?

CACHING · 5

What does this print?

```
cache = {"42": "shipped"}
print(cache.get("43", "miss"))
```

CACHING · 6

What is **read-through**?

CACHING · 7

90 % of reads hit at 2 ms, the rest take 40 ms. What is the average?

CACHING · 8

What is a **stale** value?

CACHING · 2 · ANSWER

A read the cache cannot answer; it goes on to the store behind it.

CACHING · 1 · ANSWER

A read the cache answers itself; the database is not asked.

CACHING · 4 · ANSWER

A safety net: a delete that was missed leaves a value wrong for minutes, not forever.

CACHING · 3 · ANSWER

How long a value may live in the cache before it is dropped.

CACHING · 6 · ANSWER

On a miss the value is loaded from the store and kept, so the next read hits.

CACHING · 5 · ANSWER

`miss` — `get` returns its default on a missing key.

CACHING · 8 · ANSWER

A copy the store has moved on from without telling the cache.

CACHING · 7 · ANSWER

**5.8 ms** – 0.9 × 2 + 0.1 × 40. The few misses are most of it.

CACHING · 9

What is **cache warm-up**?

CACHING · 10

Name two ways a key is **evicted**.

CACHING · 10 · ANSWER

Least recently used (LRU) when memory is full, and expiry when its TTL runs out.

CACHING · 9 · ANSWER

Filling the cache before the traffic comes, so the first reads do not all miss.